



MANUALE TECNICO

Laboratorio Interdisciplinare B — a.a. 2025/2026
Università degli Studi dell'Insubria

Autori:
Karakus Yasemin — matricola 762746 — sede VA
Choudhry Maha Ilyas — matricola 747119 — sede VA

Indice

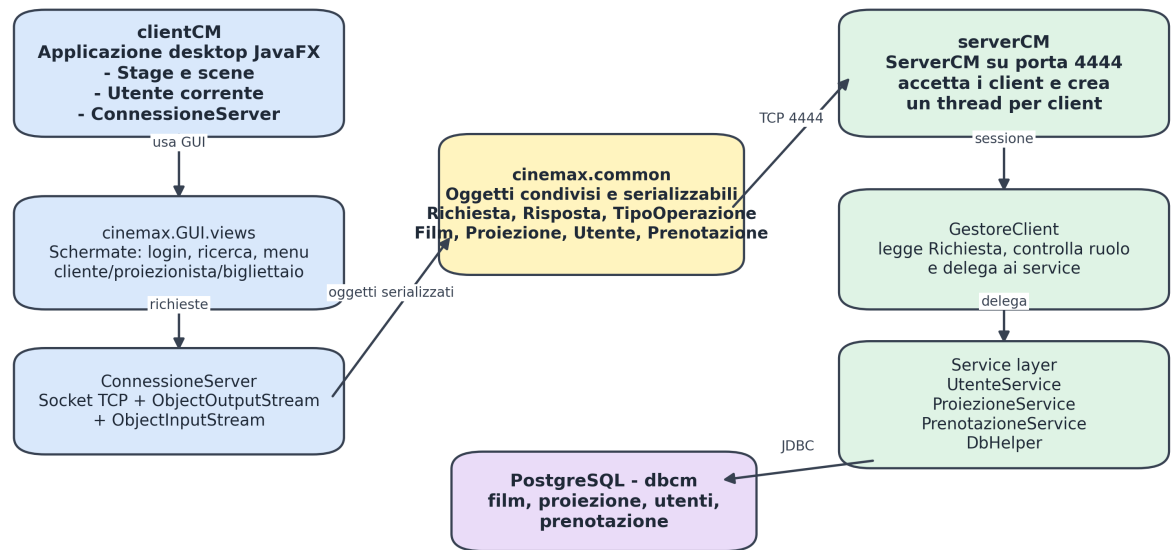
1. Introduzione, obiettivi e architettura generale
2. Tecnologie, build Maven ed esecuzione
3. Struttura del codice e protocollo client/server
4. Database, concorrenza, sicurezza e gestione errori
5. Descrizione completa delle classi e delle funzioni/metodi principali
6. Operazioni del protocollo e note di collaudo

1. Introduzione e architettura generale

CineMax è una piattaforma client/server per la gestione di un cinema monosala da 200 posti. Il sistema permette ai clienti di cercare le proiezioni e prenotare posti, ai proiezionisti di gestire il palinsesto e ai bigliettai di consultare le prenotazioni.

L'architettura è distribuita: il modulo clientCM fornisce l'interfaccia JavaFX, il modulo serverCM riceve le richieste via socket TCP e interagisce con PostgreSQL. Le classi condivise del package cinemax.common sono serializzabili e vengono usate come oggetti di scambio.

CineMax - Architettura generale client/server



Idea chiave: client grafico e server sono due JAR separati; il database resta lato server.

2. Tecnologie e build

Elemento	Scelta progettuale
Linguaggio	Java 21
Interfaccia grafica	JavaFX 21.0.5
Build	Apache Maven, packaging JAR
Database	PostgreSQL, database dbcm
Accesso ai dati	JDBC con PreparedStatement
Comunicazione	Socket TCP sulla porta 4444 con ObjectInputStream/ObjectOutputStream
Sicurezza password	BCrypt tramite estensione pgcrypto

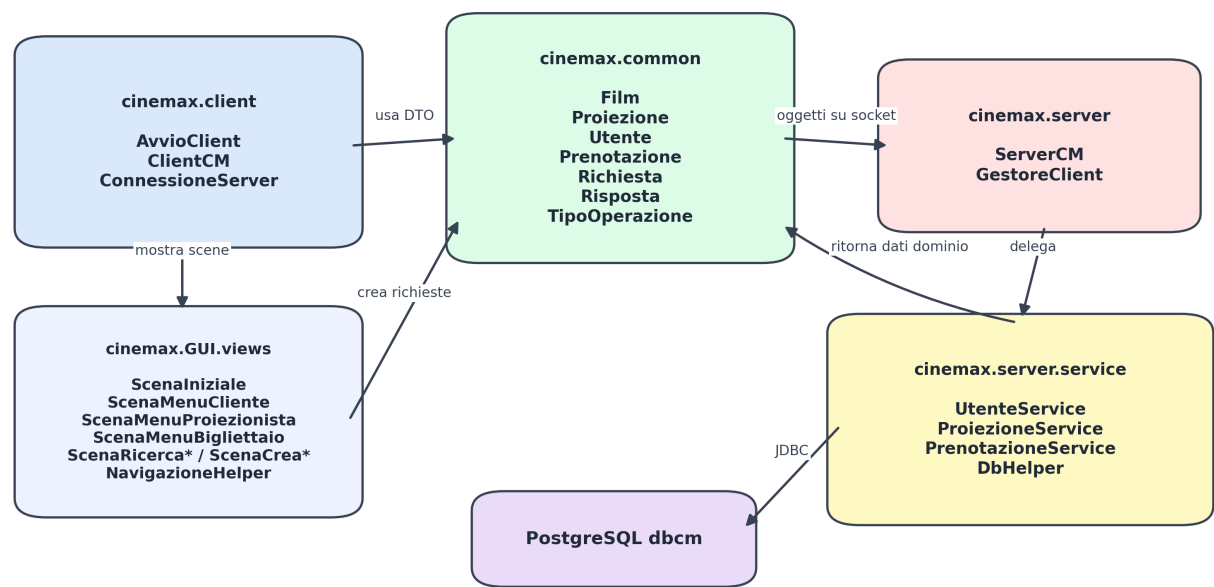
Il pom.xml definisce le dipendenze PostgreSQL JDBC e JavaFX e usa maven-shade-plugin per generare due JAR eseguibili separati: serverCM.jar e clientCM.jar.

```
mvn clean package
mvn javadoc:javadoc
java -jar target/serverCM.jar
java -jar target/clientCM.jar
```

3. Struttura dei package

Package	Responsabilita
cinemax.common	Oggetti dominio e protocollo: Film, Proiezione, Utente, Prenotazione, Richiesta, Risposta, TipoOperazione.
cinemax.client	Launcher client, applicazione JavaFX e connessione TCP verso il server.
cinemax.GUI.views	Schermate JavaFX e helper di navigazione.
cinemax.server	Avvio server, accettazione client e thread di sessione.
cinemax.server.service	Logica applicativa e accesso al database.

CineMax - Diagramma dei package e classi principali



4. Protocollo client/server

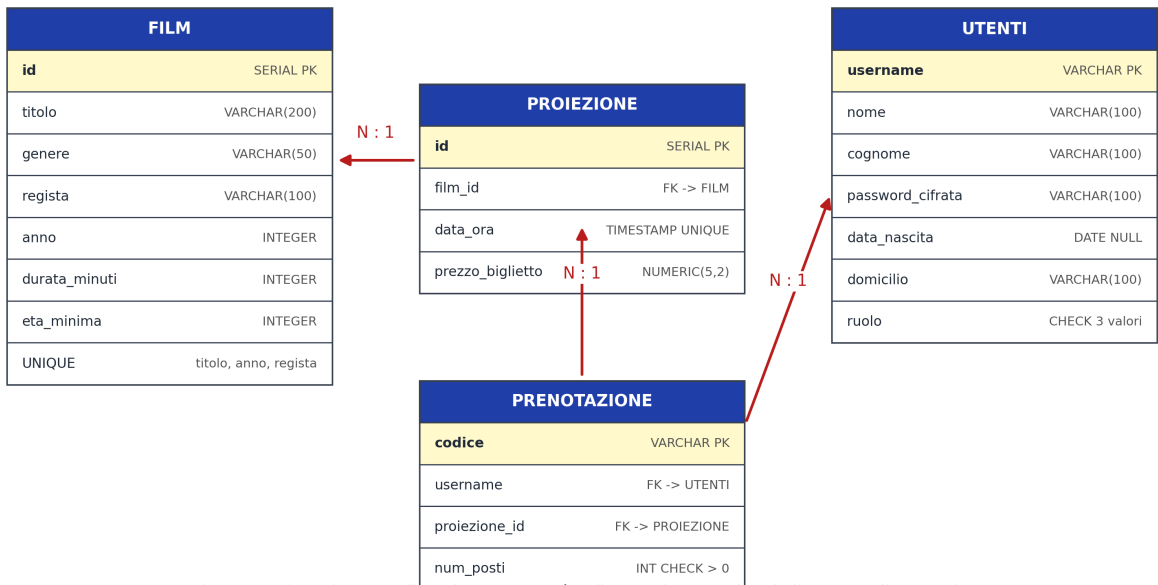
Ogni azione della GUI viene trasformata in una Richiesta. La richiesta contiene un TipoOperazione e una mappa di parametri. Il server la legge, controlla l'autorizzazione, chiama il service corretto e restituisce una Risposta con esito, messaggio e dati.

Classe	Ruolo nel protocollo
Richiesta	Trasporta operazione e parametri dal client al server.
TipoOperazione	Enum che identifica l'azione da eseguire.
Risposta	Trasporta esito, messaggio e dati dal server al client.
Film/Proiezione/Prenotazione/Utente	Oggetti dominio inviati nei dati della risposta.

5. Database e progettazione dati

Il database è composto da quattro tabelle principali: film, proiezione, utenti e prenotazione. La generalizzazione UTENTE -> CLIENTE/PROIEZIONISTA/BIGLIETTAIO è collassata nella tabella utenti tramite l'attributo ruolo.

CineMax - Schema relazionale ristrutturato (3NF)



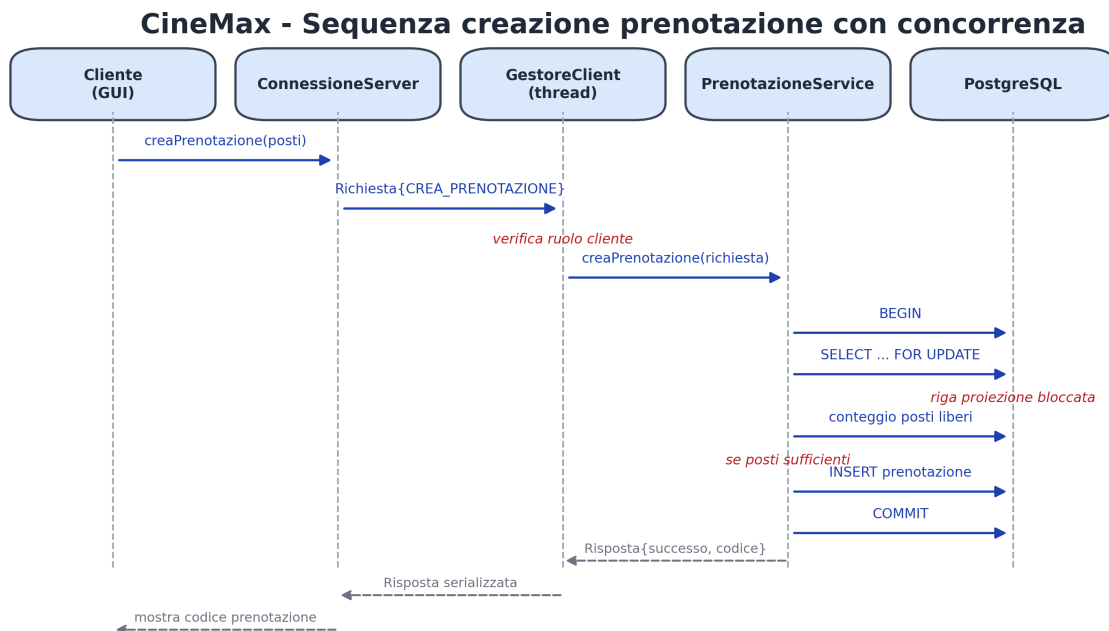
Ristrutturazione: la generalizzazione UTENTE è collassata in una sola tabella con attributo ruolo.
Tutte le tabelle sono in 3NF: gli attributi non chiave dipendono dalla chiave della tabella.

Tabella	Descrizione
film	Contiene i dati del film. Chiave naturale protetta da UNIQUE(titolo, anno, regista).
proiezione	Contiene film_id, data_ora e prezzo_biglietto. Il prezzo appartiene alla singola proiezione.
utenti	Contiene utenti di tutti i ruoli, con password cifrata e CHECK sul ruolo.
prenotazione	Contiene codice, username, proiezione_id e num_posti.

6. Concorrenza e sicurezza

Il server usa il modello thread-per-client: ServerCM accetta le connessioni e crea un GestoreClient per ciascun client. Ogni thread apre una connessione JDBC dedicata. Le operazioni critiche sulle prenotazioni usano transazioni e SELECT FOR UPDATE per evitare overbooking.

Le password sono cifrate con BCrypt tramite pgcrypto. In fase di login la verifica avviene con password_cifrata = crypt(?, password_cifrata), evitando il confronto lato Java e il salvataggio di password in chiaro.



7. Funzioni e metodi - package cinemax.common

Questa sezione contiene le funzioni/metodi principali delle classi condivise. Sono inclusi anche getter e metodi di supporto perché vengono usati dalla GUI, dal server e dalla serializzazione.

Classe	Funzione/metodo	Spiegazione
Film	Costruttore Film(...)	Inizializza un film con id, titolo, genere, regista, anno, durata e eta minima.
Film	getId(), getTitolo(), getGenere(), getRegista(), getAnno(), getDurataMinuti(), getEtaMinima()	Metodi getter che restituiscono i singoli attributi del film. Servono a popolare tabelle, dettagli proiezione e mapping dal database.
Film	toString()	Restituisce una stringa descrittiva del film con titolo, anno e regista.
Proiezione	Costruttore Proiezione(...)	Crea una proiezione con id, film associato, data/ora, prezzo del biglietto e posti liberi calcolati dal server.
Proiezione	getId(), getFilm(), getDataOra(), getPrezzoBiglietto(), getPostiLiberi()	Restituiscono le informazioni della proiezione, usate da ricerche, dettagli, menu proiezionista e prenotazioni.
Proiezione	toString()	Produce una rappresentazione testuale con titolo del film, data/ora e prezzo.
Prenotazione	Costruttore Prenotazione(...)	Inizializza una prenotazione con codice, cliente, proiezione e numero posti.
Prenotazione	getCodice(), getUsername(), getNomeCliente(), getCognomeCliente(), getProiezione(), getNumPosti()	Getter usati per visualizzare le prenotazioni nelle tabelle del cliente e del biglietto.
Prenotazione	getCostoTotale()	Calcola il costo totale moltiplicando il prezzo del biglietto per il numero di posti.
Prenotazione	toString()	Restituisce una stringa con codice, nome cliente e numero posti.
Utente	Costruttore Utente(...)	Crea l'oggetto utente senza password. La password non viene mai inviata dal server al client.
Utente	getUsername(), getNome(), getCognome(), getDataNascita(), getDomicilio(), getRuolo()	Getter per ottenere i dati anagrafici e il ruolo dell'utente autenticato.
Utente	toString()	Restituisce nome, cognome e ruolo in forma leggibile.
Richiesta	Costruttore Richiesta(TipoOperazione)	Crea una richiesta impostando l'operazione da inviare al server e inizializzando la mappa dei parametri.
Richiesta	getOperazione()	Restituisce il tipo di operazione richiesto, usato dal server per smistare la richiesta.
Richiesta	setParametro(nome, valore)	Aggiunge un parametro alla richiesta. Il valore deve essere serializzabile.
Richiesta	getParametro(nome)	Restituisce un parametro dalla mappa nome-valore.
Risposta	Costruttore Risposta(successo, messaggio, dati)	Crea la risposta del server con esito, messaggio e dati opzionali.
Risposta	isSuccesso(), getMessaggio(), getDati()	Consentono al client di sapere se l'operazione è riuscita, quale messaggio mostrare e quali dati visualizzare.
TipoOperazione	Valori enum	Elenca tutte le operazioni del protocollo: LOGIN, registrazione, ricerca proiezione, gestione prenotazioni, gestione palinsesto e logout.

8. Funzioni e metodi - clientCM

Classe	Funzione/metodo	Spiegazione
AvvioClient	main(String[] args)	Punto di ingresso del client. Delega l'avvio alla classe ClientCM per evitare problemi di avvio JavaFX nei JAR auto-contenuti.
ClientCM	start(Stage stage)	Metodo JavaFX iniziale: configura finestra, splash screen e tentativo automatico di connessione a localhost:4444.
ClientCM	mostraScena(Scene scena)	Sostituisce la scena mostrata nello Stage principale. Usato da tutte le schermate per navigare.
ClientCM	getConnessione(), setConnessione(...)	Leggono e impostano la connessione TCP attiva con il server.
ClientCM	getUtenteCorrente(), setUtenteCorrente(...)	Leggono e impostano l'utente autenticato. Con valore null si rappresenta guest o logout.
ClientCM	stop()	Alla chiusura dell'applicazione chiude la connessione con il server, se presente.
ClientCM	main(String[] args)	Invoca launch(args) e avvia il ciclo di vita JavaFX.
ConnessioneServer	Costruttore ConnessioneServer(host, porta)	Apri il socket TCP verso il server e crea ObjectOutputStream prima di ObjectInputStream per evitare deadlock.
ConnessioneServer	invia(Richiesta richiesta)	Metodo synchronized che invia una richiesta serializzata, attende una Risposta e la restituisce. Usa reset() per evitare cache di oggetti obsoleti.
ConnessioneServer	chiudi()	Chiude il socket in modo best-effort, ignorando errori di chiusura.

9. Funzioni e metodi - interfaccia grafica JavaFX

Le schermate JavaFX costruiscono la GUI e trasformano le azioni utente in Richiesta inviate al server. Le chiamate di rete sono eseguite in Task per non bloccare l'interfaccia.

Classe	Funzione/metodo	Spiegazione
ScenaConnessione	costruisci()	Crea la schermata per inserire host e porta del server. Avvia la connessione in un Task in background.
ScenaIniziale	costruisci()	Crea menu iniziale con login, accesso guest, registrazione e configurazione server.
ScenaIniziale	eseguiLogin(username, password, lblErrore, btnAccedi)	Valida le credenziali, invia LOGIN al server e apre il menu in base al ruolo restituito.
ScenaRegistrazione	costruisci()	Crea il form di registrazione cliente, valida campi obbligatori e conferma password, invia REGISTRA_CLIENTE.
ScenaRicercaProiezioni	costruisci()	Crea la ricerca proiezioni per guest e clienti. I filtri sono titolo, genere, date e prezzo.
ScenaDettaglioProiezione	costruisci()	Mostra caratteristiche film, data/ora, prezzo e posti liberi. Se l'utente è cliente permette di prenotare.
ScenaDettaglioProiezione	aggiungiRiga(grid, chiave, valore, riga)	Helper grafico per aggiungere righe etichetta-valore al GridPane.
ScenaCreaPrenotazione	costruisci()	Mostra riepilogo proiezione, selezione numero posti e invia CREA_PRENOTAZIONE.
ScenaCreaPrenotazione	aggiungiRiga(grid, chiave, valore, riga)	Helper per comporre il riepilogo della prenotazione.
ScenaMenuCliente	costruisci()	Crea menu cliente con tabella prenotazioni attive, ricerca proiezioni, aggiornamento, modifica e cancellazione.
ScenaMenuCliente	mostraDialogModifica(prenotazione, dopoOperazione, lblStatus)	Apre un dialogo per scegliere la nuova data e invia MODIFICA_PRENOTAZIONE.
ScenaMenuCliente	mostraDialogCancella(prenotazione, dopoOperazione, lblStatus)	Mostra conferma e invia ELIMINA_PRENOTAZIONE.
ScenaMenuCliente	eseguiLogout()	Invia LOGOUT, azzera l'utente corrente e torna al menu iniziale.
ScenaMenuProiezionista	costruisci()	Crea menu proiezionista con palinsesto futuro/storico, aggiunta, modifica ed eliminazione proiezioni.
ScenaMenuProiezionista	mostraDialogModifica(proiezione, dopoOperazione, lblStatus)	Permette di cambiare data e ora della proiezione selezionata, inviando MODIFICA_PROIEZIONE.
ScenaMenuProiezionista	mostraDialogElimina(proiezione, dopoOperazione, lblStatus)	Chiede conferma e invia ELIMINA_PROIEZIONE.
ScenaMenuProiezionista	eseguiLogout()	Gestisce il logout del proiezionista.
ScenaAggiungiProiezione	costruisci()	Crea form per inserire dati film, data, ora e prezzo della nuova proiezione. Invia AGGIUNGI_PROIEZIONE.
ScenaMenuBigliettaio	costruisci()	Crea menu bigliettotaio con tab prenotazioni di oggi e tab ricerca prenotazioni.
ScenaMenuBigliettaio	costruisciTabella()	Costruisce la TableView delle prenotazioni con codice, cliente, film, data, posti e totale.
ScenaMenuBigliettaio	eseguiLogout()	Gestisce il logout del bigliettotaio.
ScenaRicercaPrenotazioni	costruisciPannello()	Crea pannello di ricerca prenotazioni per codice, nome, cognome, titolo e date.
ScenaRicercaPrenotazioni	costruisci()	Racchiude il pannello in una scena autonoma, utile se aperta separatamente.
NavigazioneHelper	tornaAlMenu(Utente utente)	Riporta guest, cliente, proiezionista o bigliettotaio alla schermata corretta in base al ruolo.
NavigazioneHelper	creaBottoneHome(Utente utente)	Crea un pulsante Home già collegato alla navigazione corretta.

10. Funzioni e metodi - server e service

Questa sezione descrive il nucleo del back-end: accettazione dei client, autorizzazione per ruolo, business logic e accesso al database.

Classe	Funzione/metodo	Spiegazione
ServerCM	main(String[] args)	Legge host e credenziali del database, verifica la connessione a dbcm, apre ServerSocket sulla porta 4444 e crea un thread GestoreClient per ogni client.
GestoreClient	Costruttore GestoreClient(socket, urlJdbc, dbUser, dbPassword)	Memorizza socket e credenziali del database per la sessione client.
GestoreClient	run()	Apri connessione JDBC, crea i service, riceve Richiesta in ciclo e invia Risposta. Termina alla disconnessione del client.
GestoreClient	elabora(Richiesta richiesta)	Smista la richiesta in base al TipoOperazione e applica i controlli di autorizzazione per ruolo.
GestoreClient	haRuolo(String ruolo)	Verifica se l'utente autenticato nella sessione ha il ruolo necessario.
GestoreClient	chiudiRisorse()	Chiude connessione JDBC e socket in modo best-effort.
UtenteService	Costruttore UtenteService(Connection conn)	Riceve la connessione JDBC della sessione.
UtenteService	login(Richiesta richiesta)	Verifica username e password con crypt() di PostgreSQL; se corrette restituisce un oggetto Utente senza password.
UtenteService	registraCliente(Richiesta richiesta)	Inserisce un nuovo cliente nel database cifrando la password con BCrypt tramite gen_salt('bf').
ProiezioneService	Costruttore ProiezioneService(Connection conn)	Riceve la connessione JDBC della sessione.
ProiezioneService	cercaProiezione(Richiesta richiesta)	Costruisce dinamicamente la WHERE in base ai filtri titolo, genere, date e prezzo; restituisce lista di Proiezione.
ProiezioneService	visualizzaProiezione(Richiesta richiesta)	Recupera una proiezione specifica per id con conteggio posti liberi aggiornato.
ProiezioneService	aggiungiProiezione(Richiesta richiesta)	Trova o crea il film, verifica sovrapposizioni temporali e inserisce la nuova proiezione in transazione.
ProiezioneService	modificaProiezione(Richiesta richiesta)	Modifica data/ora di una proiezione solo se non ha prenotazioni e non si sovrappone ad altre.
ProiezioneService	eliminaProiezione(Richiesta richiesta)	Elimina una proiezione solo se non esistono prenotazioni collegate.
ProiezioneService	visualizzaPalinsesto(Richiesta richiesta)	Restituisce proiezioni future o storiche ordinate per data.
PrenotazioneService	Costruttore PrenotazioneService(Connection conn)	Riceve la connessione JDBC della sessione.
PrenotazioneService	creaPrenotazione(Richiesta richiesta)	Usa transazione e SELECT FOR UPDATE sulla proiezione, verifica posti liberi, genera codice UUID abbreviato e inserisce la prenotazione.
PrenotazioneService	visualizzaPrenotazioni(Richiesta richiesta)	Restituisce le prenotazioni future di un cliente.
PrenotazioneService	modificaPrenotazione(Richiesta richiesta)	Sposta una prenotazione a una nuova data dello stesso film, solo se date future e posti disponibili. Usa lock FOR UPDATE.
PrenotazioneService	eliminaPrenotazione(Richiesta richiesta)	Cancella una prenotazione solo se la relativa proiezione è futura.
PrenotazioneService	cercaPrenotazione(Richiesta richiesta)	Per il bigliettaio, cerca prenotazioni per codice, nome, cognome, titolo e intervallo date.
PrenotazioneService	prenotazioniOggi()	Restituisce le prenotazioni della data corrente ordinate per orario.
DbHelper	SQL_PROIEZIONE	Frammento SQL condiviso per recuperare proiezioni, film e posti liberi calcolati.
DbHelper	SQL_PRENOTAZIONE	Frammento SQL condiviso per recuperare prenotazioni con dati utente, film, proiezione e posti liberi.
DbHelper	proiezioneFromRs(ResultSet rs)	Converte una riga del ResultSet in un oggetto Proiezione contenente anche il Film associato.
DbHelper	prenotazioneFromRs(ResultSet rs)	Converte una riga del ResultSet in un oggetto Prenotazione completo.

11. Operazioni del protocollo TipoOperazione

Operazione	Significato
LOGIN	Accesso utente registrato.
REGISTRA_CLIENTE	Registrazione di un nuovo cliente.
CERCA_PROIEZIONE	Ricerca proiezioni con filtri opzionali.
VISUALIZZA_PROIEZIONE	Dettaglio di una proiezione.
CREA_PRENOTAZIONE	Creazione prenotazione cliente.
VISUALIZZA_PRENOTAZIONI	Elenco prenotazioni del cliente.
MODIFICA_PRENOTAZIONE	Cambio data prenotazione.
ELIMINA_PRENOTAZIONE	Cancellazione prenotazione futura.
CERCA_PRENOTAZIONE	Ricerca prenotazioni per bigliettotaio.
PRENOTAZIONI_OGGI	Prenotazioni della giornata per bigliettotaio.
AGGIUNGI_PROIEZIONE	Aggiunta proiezione per proiezionista.
MODIFICA_PROIEZIONE	Modifica data/ora proiezione.
ELIMINA_PROIEZIONE	Eliminazione proiezione senza prenotazioni.
VISUALIZZA_PALINSESTO	Palinsesto futuro o storico.
LOGOUT	Chiusura sessione utente.

12. Gestione degli errori

- La GUI valida campi obbligatori, formati numerici, date e conferma password prima dell'invio.
- Il server restituisce Risposta(false, messaggio, null) per errori applicativi come credenziali errate, posti insufficienti, ruolo non autorizzato o sovrapposizione proiezioni.
- Il database applica vincoli dichiarativi: primary key, foreign key, unique e check.
- Le eccezioni di rete sono intercettate nei Task JavaFX e mostrate all'utente tramite label o dialoghi.

13. Collaudo consigliato

Test	Risultato atteso
Login test/test	Apertura menu cliente e visualizzazione prenotazioni attive.
Ricerca guest	Visualizzazione proiezioni compatibili con il titolo inserito.
Creazione prenotazione	Generazione codice prenotazione e aggiornamento posti liberi.
Bigliettaio - ricerca prenotazioni	Ritrovamento della prenotazione per codice o dati cliente.
Proiezionista - aggiunta proiezione	Inserimento se non sovrapposta, rifiuto se sovrapposta.
Proiezionista - eliminazione proiezione con prenotazioni	Rifiuto dell'operazione con messaggio descrittivo.
Due client contemporanei	Nessun overbooking grazie a SELECT FOR UPDATE.

14. Nota sul livello di dettaglio del manuale

Il manuale tecnico contiene sia le scelte architetturali sia una descrizione delle classi e dei metodi. Per evitare duplicazioni inutili, non riporta riga per riga il codice sorgente: il codice completo è documentato tramite commenti Javadoc. Le tabelle precedenti spiegano però il ruolo delle funzioni principali e dei metodi pubblici/privati nominati nelle classi del progetto.